Posts and Telecommunications Institute of Technology



Object-Oriented Programming (OOP)
Chapter 1

Faculty: Information Technology 1

Department: Computer Science

Instructor: Pham Hoang Viet

Email: vietph@ptit.edu.vn **Phone:** 0945296388

Chapter 1

Introduction to the Java Programming Language

COURSE MAP

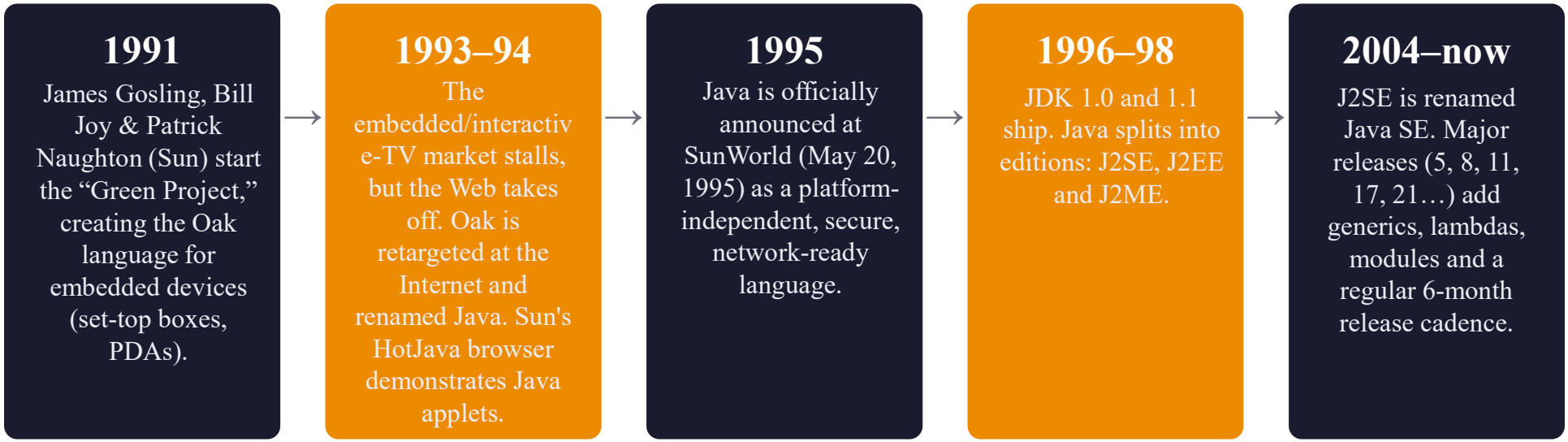
Learning outcomes

- History of Java
- Basic IDEs
- Compiling & Running
- Basic Data Types
- Basic Control Flows
- Standard Input/Output

1.1. History of Java

HISTORY OF JAVA

From Oak to Java: A Brief Timeline



Key takeaway: Java's original design goals — platform independence, reliability, and a compact footprint — were set for embedded devices in 1991, but turned out to be exactly what the Web (and later, the enterprise, mobile, and cloud) needed.

What Made Java's Design Different

S

Simple

Familiar C/C++-like syntax, but removes pointers, operator overloading and manual memory management.

O

Object-Oriented

Everything (besides primitives) is an object; supports encapsulation, inheritance and polymorphism.

N

Distributed

Rich networking libraries make it easy to work with URLs, sockets and remote resources.

R

Robust

Strong compile-time and run-time checking; no explicit pointer arithmetic; automatic garbage collection.

Se

Secure

Bytecode verification and a sandboxed execution model were built in from the start.

A

Architecture-Neutral & Portable

Compiles to bytecode that runs unmodified on any device with a JVM — “write once, run anywhere.”

H

High-Performance

Just-in-time (JIT) compilation translates hot bytecode to native instructions at run time.

M

Multithreaded

Concurrency is a first-class citizen, with built-in support for multiple threads of execution.

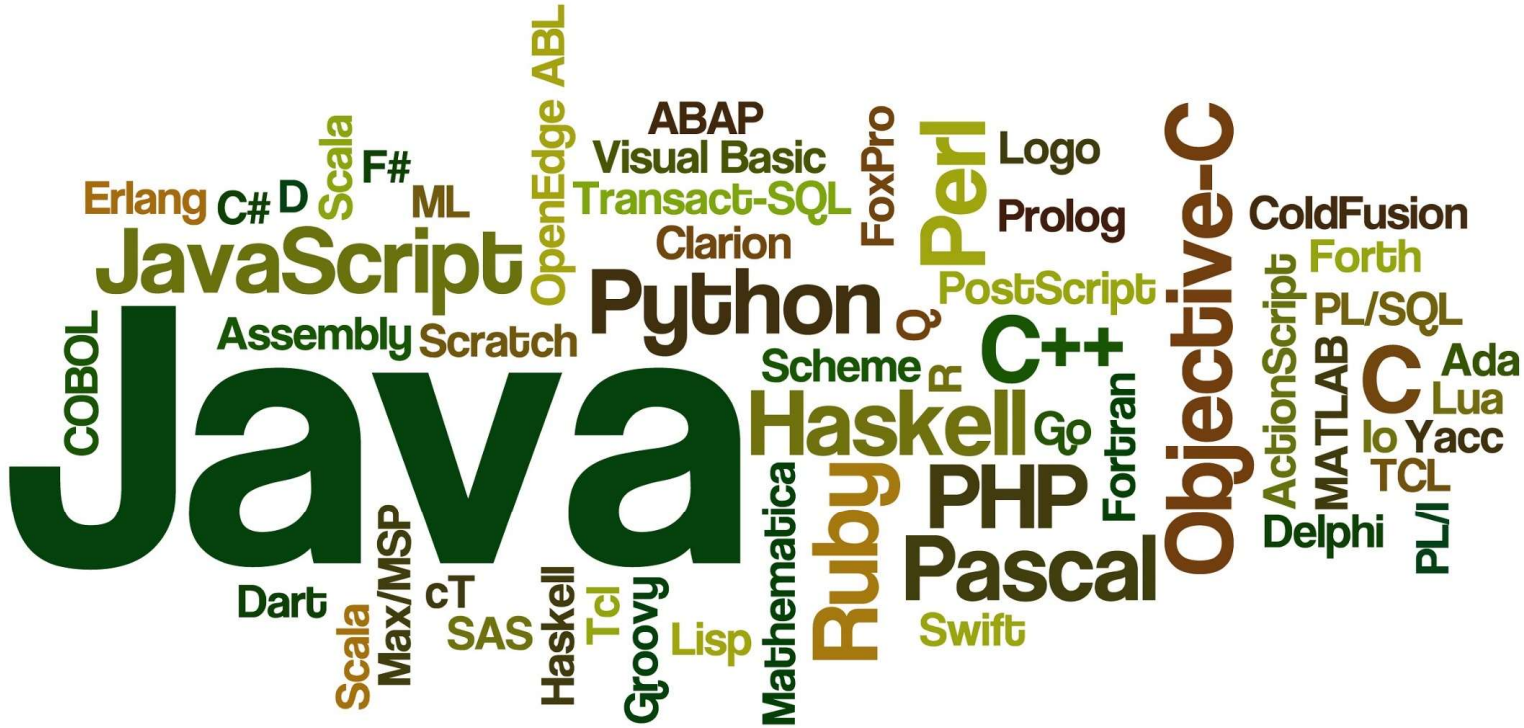
D

Dynamic

Classes are loaded on demand at run time, allowing programs to adapt to an evolving environment.

WHY JAVA?

To this day, Java remains a popular programming language.



Top ten common programming languages:

Jul 2018	Jul 2017	Change	Programming Language	Ratings	Change
1	1		Java	16.139%	+2.37%
2	2		C	14.662%	+7.34%
3	3		C++	7.615%	+2.04%
4	4		Python	6.361%	+2.82%
5	7	^	Visual Basic .NET	4.247%	+1.20%
6	5	v	C#	3.795%	+0.28%
7	6	v	PHP	2.832%	-0.26%
8	8		JavaScript	2.831%	+0.22%
9	-	^^	SQL	2.334%	+2.33%
10	18	^^	Objective-C	1.453%	-0.44%

From http://www.tiobe.com/index.php/content/paperinfo/tpci/index.html

Jul 2026	Jul 2025	Change	Programming Language	Ratings	Change
1	1		Python	18.94%	-8.03%
2	3	↑	C	10.86%	+1.22%
3	2	↓	C++	9.12%	-0.68%
4	4		Java	8.03%	-0.73%
5	5		C#	4.49%	-0.38%
6	6		JavaScript	2.72%	-0.63%
7	8	↑	Visual Basic	2.48%	+0.54%
8	13	⇈	SQL	1.71%	+0.32%
9	15	⇈	R	1.69%	+0.44%
10	18	⇈	Rust	1.34%	+0.33%

What Can You Build With Java Today?



Desktop Applications

Console tools and GUI apps (JavaFX, Swing).



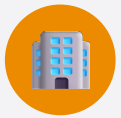
Web Applications

Servlets, JSP, and modern frameworks like Spring.



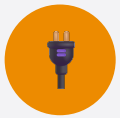
Mobile

Native Android applications are written in Java or Kotlin on the JVM.



Enterprise Systems

Banking, ERP, and large-scale backend services (Java EE / Jakarta EE).



Embedded Systems

Firmware for devices with constrained memory and processing power.



Cloud & Distributed Systems

Microservices, big-data engines (Hadoop, Spark run on the JVM).

Why Java for OOP?

- ❖Java is designed with OOP as a core principle.
- ❖OOP concepts are clearly demonstrated in Java.
- ❖Java features static typing, helping students clearly visualize the design.
- ❖Java compels students to pay attention to encapsulation.
- ❖Java interfaces are excellent for teaching abstraction and loose coupling.
- ❖Java helps students transition from OOP to Software Engineering.

Java provides	What we learn
Classes & Objects	Object modeling
Access modifiers	Encapsulation
Inheritance	Code reuse & specialization
Abstract classes	Abstraction
Interfaces	Loose coupling
Method overriding	Polymorphism
Static typing	Type relationships
Exceptions	Robust software
Collections & Generics	Reusable components
JUnit	Software testing
Rich ecosystem	Real-world development

JDK Editions: Choosing the Right Java for the Job

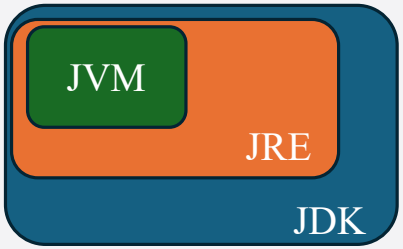
Edition	Full Name	Typical Use
Java SE	Java Standard Edition (formerly J2SE)	Client-side standalone applications and applets — the foundation every other edition builds on.
Java EE	Java Enterprise Edition (formerly J2EE)	Server-side applications: Java Servlets, JavaServer Pages, large multi-tier business systems.
Java ME	Java Micro Edition (formerly J2ME)	Resource-constrained devices: feature phones, embedded controllers, some IoT devices.

- This course focuses entirely on Java SE — the edition every other edition, and every Java IDE, is built on top of.

1.2. Basic IDEs

JDK, JRE, and JVM: Know the Difference

- ❖ **JVM (Java Virtual Machine):** an abstract computing machine that executes Java bytecode (.class files). It's what makes a program portable.
- ❖ **JRE (Java Runtime Environment):** the JVM plus the core class libraries needed to run already-compiled Java programs.
- ❖ **JDK (Java Development Kit):** the JRE plus the development tools — most importantly the javac compiler — needed to write and build Java programs.
- ❑ **Rule of thumb:** end users only need a JRE; developers need the full JDK.



Each layer contains the one below it. As a student, you will always install the full JDK.

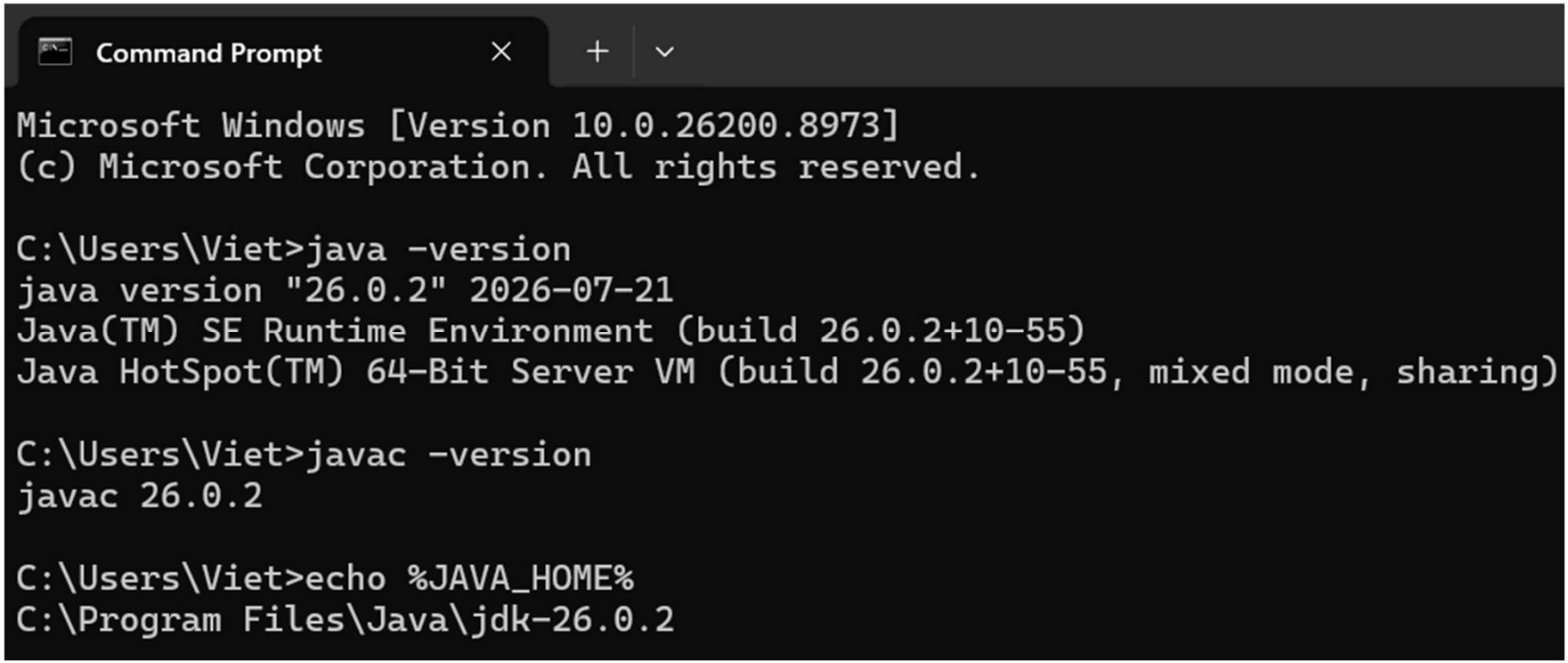
Popular Java IDEs

IDE	Highlights	Good For
Eclipse	Free, extensible via plugins, huge community, industry-standard for many enterprise teams.	General-purpose Java & Java EE development.
NetBeans (Apache NetBeans)	Official Oracle-recommended IDE for learning; simple project wizard; built-in GUI builder.	Beginners and academic coursework (used in this course).
IntelliJ IDEA	Very strong code completion and refactoring tools; Community edition is free.	Professional development, especially Spring/Kotlin projects.
BlueJ	Minimalist, visual, designed specifically for teaching first-year OOP concepts.	Introductory OOP courses.
TextPad / plain text editor	No IDE features — you run javac and java by hand from a terminal.	Understanding what the IDE is doing “under the hood.”

Setting Up Your Environment

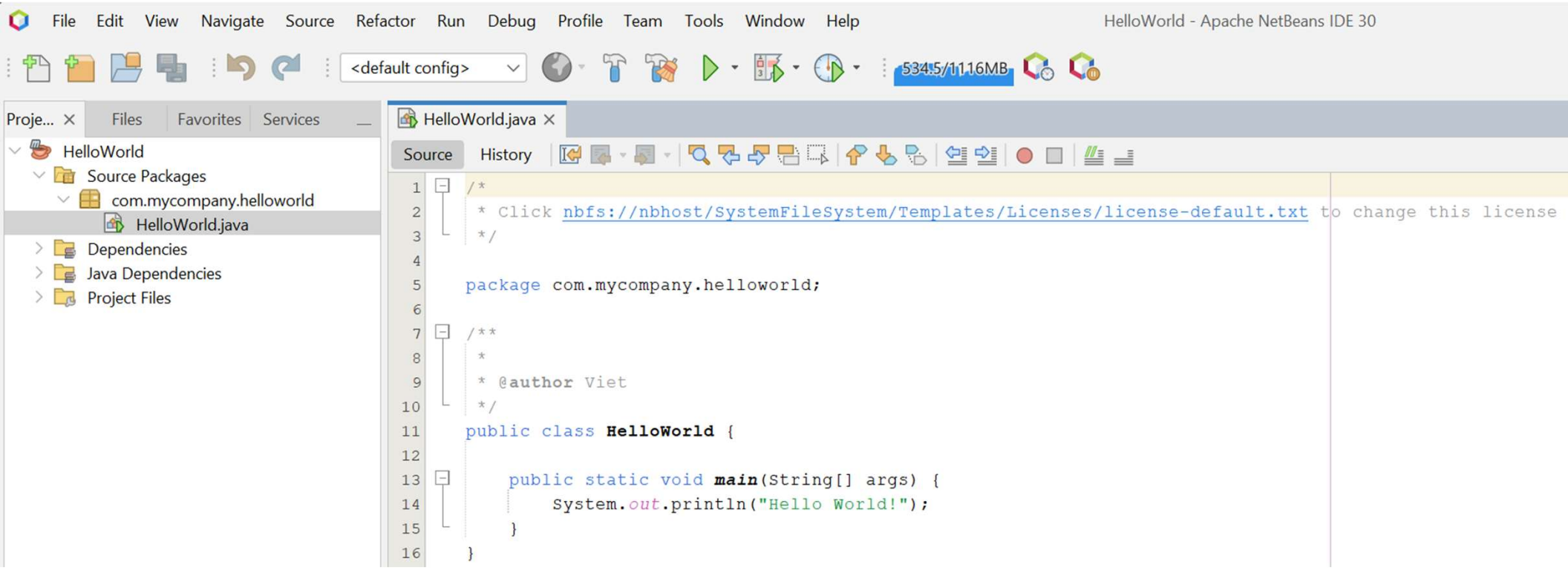
- **Step 1** — Install the JDK: download JDK 8 (or newer) from the official Oracle site (oracle.com/java/technologies/downloads) and run the installer. On Windows it is typically installed under C:\Program Files\Java, containing a jdk and (on older versions) a jre folder.
- **Step 2** — Set environment variables: add the JDK's bin folder to your system PATH so the javac and java commands are available from any terminal. On Windows: My Computer → Properties → Advanced → Environment Variables → System Variables → Path → Edit.
  - JAVA_HOME = C:\Program Files\Java\jdk-26.0.2
  - Add to Path %JAVA_HOME%\bin
- **Step 3** — Verify the installation: open a terminal and run `java -version` and `javac -version` . Both should print a matching version number.
- **Step 4** — Install an IDE: download Apache NetBeans (netbeans.apache.org) or Eclipse. During installation, make sure the IDE is pointed at the JDK you just installed (important if you have more than one JDK version on your machine).
- **Step 5** — Create your first project: in NetBeans, choose File → New Project → Java with Ant → Java Application, give it a name, and the IDE scaffolds a runnable “HelloWorld” class for you.

Step 3


Command Prompt
Microsoft Windows [Version 10.0.26200.8973]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Viet>java -version
java version "26.0.2" 2026-07-21
Java(TM) SE Runtime Environment (build 26.0.2+10-55)
Java HotSpot(TM) 64-Bit Server VM (build 26.0.2+10-55, mixed mode, sharing)
C:\Users\Viet>javac -version
javac 26.0.2
C:\Users\Viet>echo %JAVA_HOME%
C:\Program Files\Java\jdk-26.0.2

1.3. Compiling and Running a Program

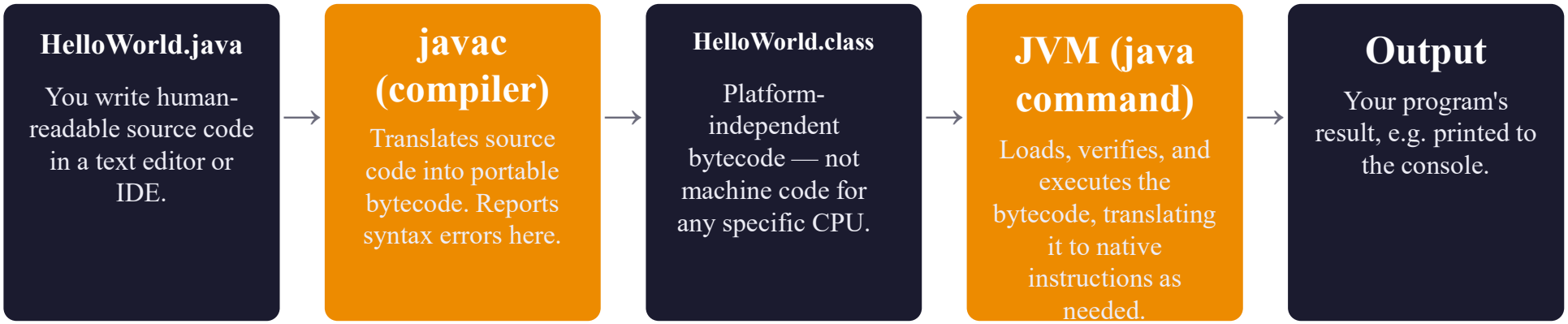
Your First Java Program



- Save this file as exactly HelloWorld.java — the file name must match the public class name.
- Compile it: javac HelloWorld.java → produces Hello.class
- Run it: java HelloWorld (note: no ".class" on the command line)
- Expected output: Hello World!

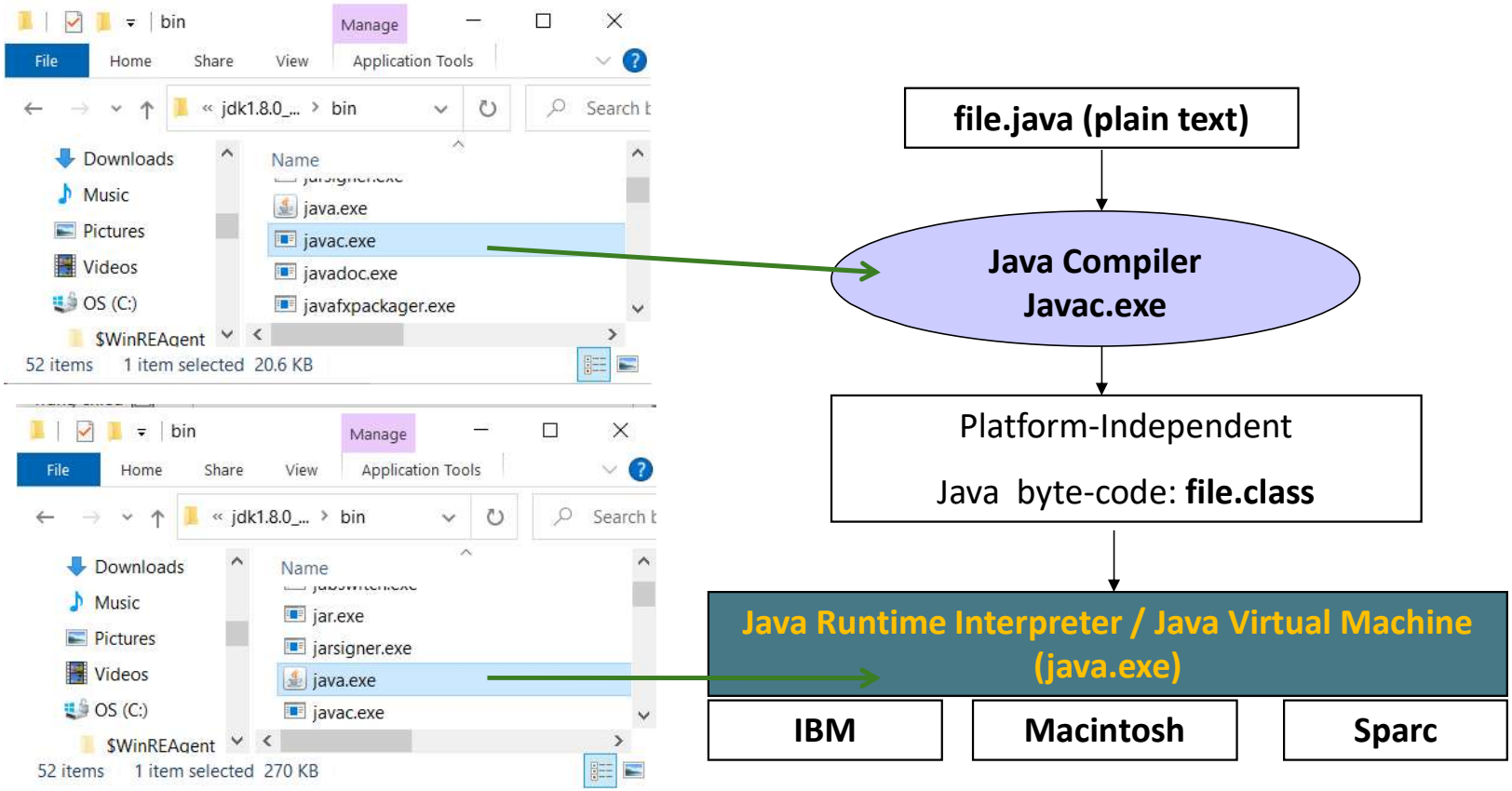
- Question to think about: what happens if you rename the file but not the class, or vice versa?

From Source Code to Running Program



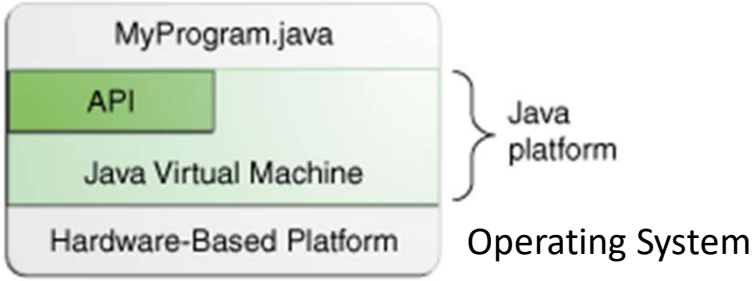
This extra bytecode step is what makes Java different from C/C++, whose compilers translate straight to machine code for one specific platform. It costs a small amount of speed, but in exchange the very same .class file runs unmodified on Windows, Linux, macOS — anywhere a JVM exists. This is Java's famous promise: “**write once, run anywhere**.”

How can Java support platform-independence?



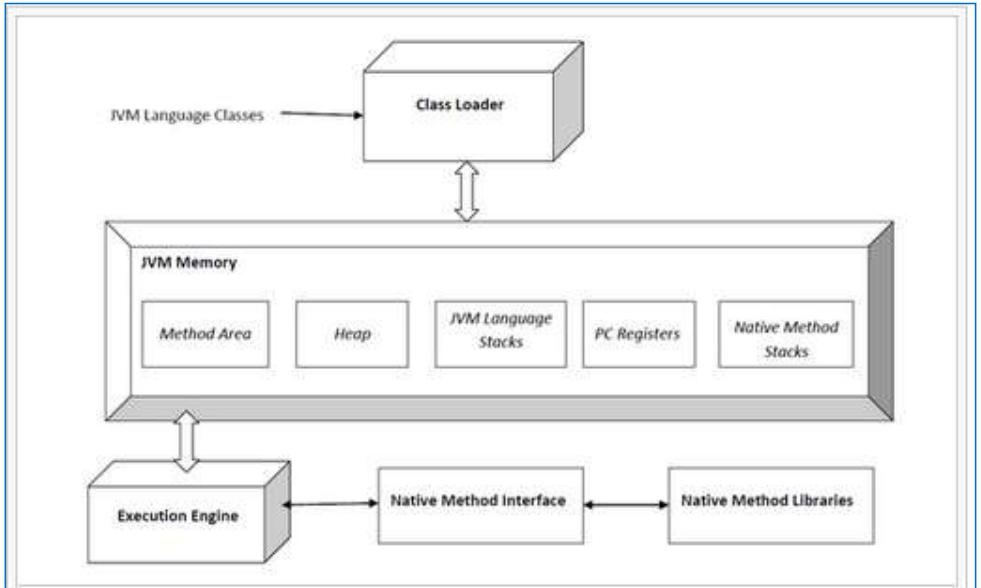
Java Platform

- A platform is the hardware or software environment in which a program runs.
- The Java platform has two components:
  - The Java Virtual Machine
  - The Java **A**pplication **P**rogramming **I**nterface (API)



Java Virtual Machine

- The Java Virtual Machine is an abstract computing machine. Like a real computing machine, it has an instruction set and manipulates various memory areas at run time. It is reasonably common to implement a programming language using a virtual machine; the best-known virtual machine may be the P-Code machine of UCSD Pascal



Overview of a Java virtual machine (JVM) architecture based on The Java Virtual Machine Specification Java SE 7 Edition

http://en.wikipedia.org/wiki/Java_virtual_machine

Anatomy of a Java Program

- **Class name:** every program needs at least one class; by convention its name starts with an uppercase letter (e.g. Hello).
- **The main method:** public static void main(String[] args) is the fixed entry point — execution always begins here.
- **Statements:** each line that performs an action, such as System.out.println(…);, ends with a semicolon.
- **Blocks:** a pair of curly braces { … } groups statements that belong together (a class body, a method body, a loop body).
- **Comments:** // single-line and /* … */ multi-line comments are ignored by the compiler but essential for readability.

```java
/**
 *
 * @author Viet
 */
public class HelloWorld {

    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

System.out.println

- System.out.println: A statement to print a line of output to the console.
  - pronounced “prɪnt laɪn”

- Two ways to use System.out.println:
  - System.out.println(**"<message>"**);
    Prints the given message as a line of text to the console.

  - System.out.println();
    Prints a blank line to the console.

Class Name

Every Java program must have at least one class. Each class has a name. By convention, class names start with an uppercase letter. In this example, the class name is Welcome.

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Main Method

Line 2 defines the main method. In order to run a class, the class must contain a method named main. The program is executed from the main method.

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Statement

A statement represents an action or a sequence of actions. The statement System.out.println("Welcome to Java!") in the program in Listing 1.1 is a statement to display the greeting “Welcome to Java!”.

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Statement Terminator

Every statement in Java ends with a semicolon (;).

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

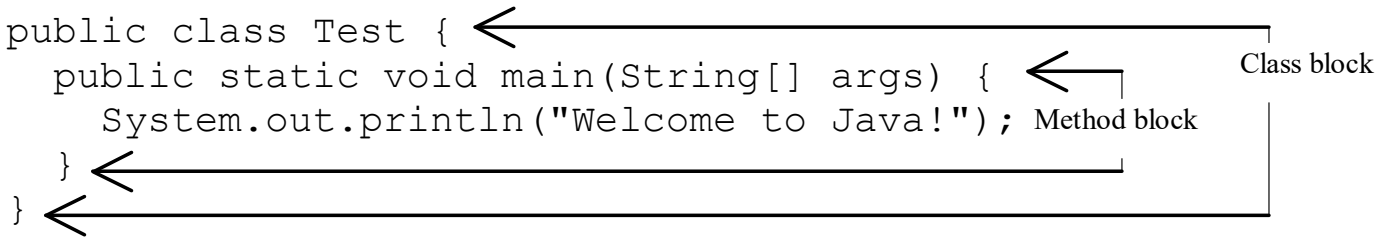
Reserved words

Reserved words or keywords are words that have a specific meaning to the compiler and cannot be used for other purposes in the program. For example, when the compiler sees the word class, it understands that the word after class is the name for the class.

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Blocks

A pair of braces in a program forms a block that groups components of a program.



Special Symbols

Character	Name	Description
{}	Opening and closing braces	Denotes a block to enclose statements.
()	Opening and closing parentheses	Used with methods.
[]	Opening and closing brackets	Denotes an array.
//	Double slashes	Precedes a comment line.
" "	Opening and closing quotation marks	Enclosing a string (i.e., sequence of characters).
;	Semicolon	Marks the end of a statement.

Command Line vs. IDE

From the Command Line

- javac HelloWorld.java — compiles the source, produces HelloWorld.class
- java HelloWorld — launches the JVM and runs the main method
- Forces you to understand exactly what each tool does
- No project files, no build configuration — just the JDK

From an IDE (e.g. NetBeans)

- Click “Run” — the IDE calls javac and java for you behind the scenes
- Integrated error highlighting before you even compile
- Debugger, auto-complete, and project/dependency management
- Recommended for coursework once you understand the underlying pipeline

Three Kinds of Programming Errors

Error Type	When It's Caught	Example
Syntax Error	At compile time, by javac.	public static main(String[] args) { … — missing the void return type.
Runtime Error	While the program is executing; it aborts.	System.out.println(1 / 0); — division by zero throws an ArithmeticException.
Logic Error	Never automatically — the program runs and produces the wrong answer.	(9 / 5) * 35 + 32 in integer arithmetic silently truncates 9/5 to 1, giving the wrong Fahrenheit conversion.

- Logic errors are the most dangerous of the three, precisely because the compiler and the JVM both stay silent — only careful testing catches them.

- Syntax Errors

```
public class ShowSyntaxErrors {
  public static main(String[] args) {
    System.out.println("Welcome to
 Java);
  }
}
```

- Runtime Errors

```
public class ShowRuntimeErrors {
  public static void main(String[] args) {
    System.out.println(1 / 0);
  }
}
```

❑ Logic Errors

```
public class ShowLogicErrors {
  public static void main(String[] args) {
    System.out.print("Celsius 35 is ");
    System.out.print("Fahrenheit ");
    System.out.println((9 / 5) * 35 + 32);
  }
}
```

Programming Style & Documentation

- Appropriate comments: open every file with a short summary of what the program does, its key data structures, and any non-obvious technique it uses.
- Header block: include your name, class/section, instructor, and date at the top of each source file you submit.
- Naming conventions: classes use UpperCamelCase and are nouns (ComputeExpression); methods and variables use lowerCamelCase and are verbs or noun phrases (computeArea, totalScore).
- Constants: written in ALL_CAPS with underscores between words (MAX_SCORE).
- Indentation & spacing: indent each nested block consistently (2 or 4 spaces); add blank lines between logical sections.
- Block style: this course uses end-of-line brace style — the opening { stays on the same line as the statement that introduces it.

Block Styles

Use end-of-line style for braces.

Next-line style

```
public class Test
{
  public static void main(String[] args)
  {
    System.out.println("Block Styles");
  }
}
```

End-of-line style

```
public class Test {
  public static void main(String[] args) {
    System.out.println("Block Styles");
  }
}
```

1.4. Basic Data Types

Primitive Types

Type	Size	Range / Values	Example
boolean	1 bit (conceptually)	true or false	boolean pass = true;
char	2 bytes	single Unicode character (\u0000 to \uFFFF)	char grade = 'A';
byte	1 byte	−128 to 127	byte b = 100;
short	2 bytes	−32,768 to 32,767	short s = 2000;
int	4 bytes	about ±2.1 billion	int age = 21;
long	8 bytes	about $\pm 9.2 \times 10^{18}$	long pop = 8_000_000_000L;
float	4 bytes	single-precision decimal	float pi = 3.14f;
double	8 bytes	double-precision decimal (default for decimals)	double gpa = 3.87;

- A value with a decimal point defaults to double unless suffixed with f/F (float) or L for a whole-number literal that needs long.

By Category

- Discrete (whole numbers): byte, short, int, long
- Continuous (decimals): float, double
- Non-numeric: boolean, char
- Every primitive holds exactly one value — never null.

Primitive vs. Reference Types

- A primitive stores its actual value directly in memory.
- A reference type (class, array, interface) stores an address pointing to an object created elsewhere — e.g. String s = new String("Hi");
- The default value of an uninitialized reference variable is null; primitives always have a defined zero-like default.

Declaring Variables & Naming Rules

- Declaration syntax: <type> <name> [= initial value];
  - e.g. int x; double myGPA = 3.6; int varName = 0;
- Identifiers must begin with a letter, $, or _, and may contain letters, digits, $, or _ after that — but must not be a reserved keyword.
- Java is case-sensitive: myScore and myscore are two different identifiers.
- Class constants: use static final so the value is shared across the class and can never change once set
  - e.g. public static final int MAX_SCORE = 100;
- Choose meaningful, descriptive names over short cryptic ones
  - totalPrice, not tp.

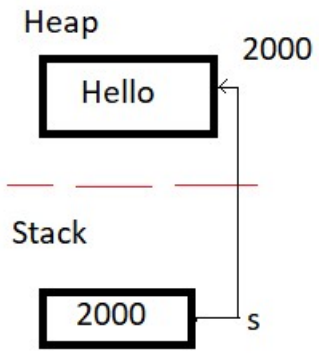
Keywords and Identifiers

- Keywords: Almost of them are similar to those in C language
- Java is a case-sensitive language
- Identifiers must be different to keywords
- Naming Convention:
  - All identifiers should begin with a letter (A to Z or a to z), currency character ($) or an underscore (_).
  - After the first character, identifiers can have any combination of characters.
  - Class and Interface should start with the uppercase letter. They should be a noun for Class and an adjective for Interface.
  - Method should start with lowercase letter, it should be a verb. If the name contains multiple words, start it with a lowercase letter followed by an uppercase letter.
  - Variable should start with a lowercase letter, it should not start with the special characters like & (ampersand), $ (dollar), _ (underscore). If the name contains multiple words, start it with the lowercase letter followed by an uppercase letter.
  - Package should be a lowercase letter.
  - Constant should be in uppercase letters. If the name contains multiple words, it should be separated by an underscore(_).

Reference Data Types - Variables

- A reference data type contains reference/address of dynamically created objects.
- reference types in Java:
  - Class types.
  - Array types
  - Interface types
- Default value of any reference variable is null

for example:
String s=new String("Hello");



Literals: Writing Values Directly in Code

- Integer literals: 28 (decimal), 0x1C (hexadecimal), 0b11100 (binary) — default type int; append L for a long literal (123L).
- Floating-point literals: 1.234 defaults to double; append f/F for a float (1.3f) or d/D to be explicit about double (1.3d).
- Character literals: single quotes, e.g. 'a', '7', or an escape sequence like '\n'.
- String literals: double quotes, e.g. "Hello"; Strings are reference types, not primitives.
- Common escape sequences: \n (newline), \t (tab), \\ (backslash), \" (double quote).
- Underscore separators (Java 7+) improve readability in long numeric literals: 1_000_000.

Operators, from Highest to Lowest Precedence

Category	Operators
Unary	++ -- + - ! ~ (type)
Arithmetic	* / % then + -
Shift	<< >> >>>
Relational	< <= > >= instanceof then == !=
Bitwise	& ^ |
Short-circuit logical	&& ||
Conditional (ternary)	condition ? valueIfTrue : valueIfFalse
Assignment	= += -= *= /= %=

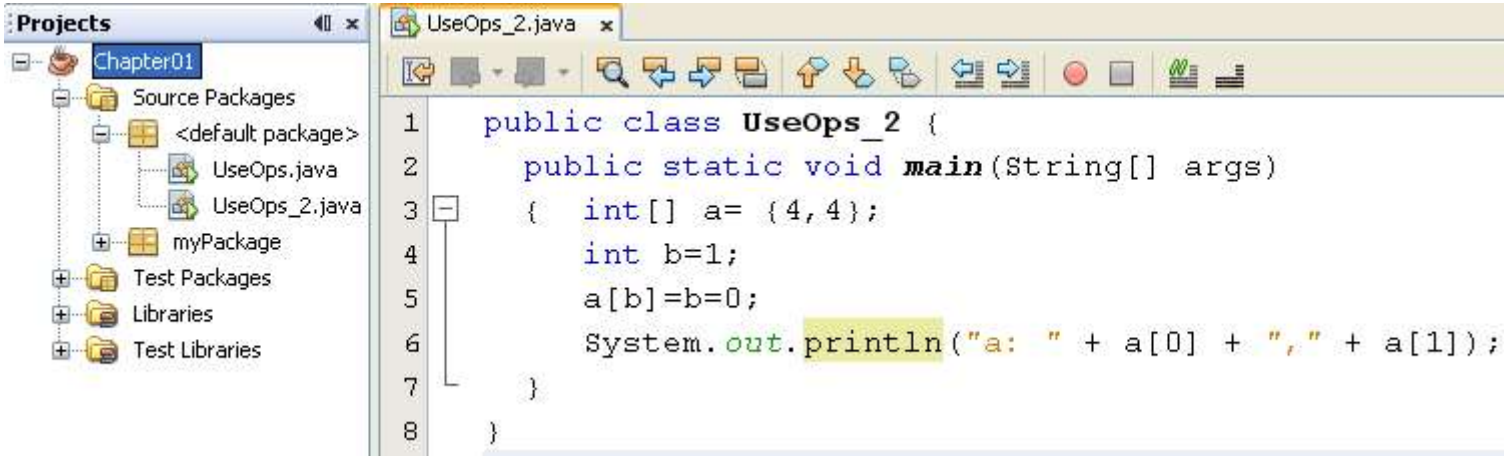
- Just like in C: the compiler evaluates an expression from the innermost parentheses outward, and left to right within the same precedence level.

❑ Evaluating Expressions and Operator Precedence

- The compiler generally evaluates such expressions from the innermost to outermost parentheses, left to right.

```
int x = 1; int y = 2; int z = 3;
int answer = ((8 * (y + z)) + y) * x;
would be evaluated piece by piece as follows:
((8 * (y + z) ) + y) * x
((8 * 5) + y) * x
(40 + y) * x
42 * x
42
```

❑ Operator Precedence- Evaluation Order



Two Classic Gotchas Every Java Programmer Hits

- **Floating-point imprecision:** System.out.println(0.1 + 0.2); prints 0.30000000000000004, because most decimal fractions cannot be represented exactly in binary floating-point.
- **Integer overflow:** int n = 2000000000; System.out.println(n * n); prints a negative number, because the true 64-bit result silently gets truncated to 32 bits, and the leftover top bit is read as a sign bit.
- **Why this matters:** these are logic errors — no compiler warning, no crash — which is exactly why understanding a type's range and precision up front is part of writing correct Java code.

❑ Overflow example

int n = 2000000000;

System.out.println(n * n);

// output: -1651507200

- the result of n*n is 4,000,000,000,000,000,000 which needs 64-bits:

---------- high-order bytes -------

00110111 10000010 11011010 11001110

---------- low order bytes --------

10011101 10010000 00000000 00000000

- In the case of overflow, Java discards the high-order bytes, retaining only the low-order ones
- In this case, the low order bytes represent 1651507200, and since the right most bit is a 1 the sign value is negative.

UseOps.java

```
1  public class UseOps {
2    public static void main(String[] args)
3    {  int x=-1;
4       System.out.println("-1<<1: " + (x<<1) );
5       System.out.println("-1>>1: " + (x>>1) );
6       System.out.println("-1>>>1: " + (x>>>1));
7       System.out.println("3|4: " + (3|4));
8       System.out.println("3&4: " + (3&4));
9       System.out.println("3^4: " + (3^4));
10      String S="Hello";
11      boolean result= S instanceof String;
12      System.out.println("Hello is an instance of String: " + result);
13   }
14 }
```

Output - Chapter01 (run)

```
run:
-1<<1: -2
-1>>1: -1
-1>>>1: 2147483647
3|4: 7
3&4: 0
3^4: 7
Hello is an instance of String: true
BUILD SUCCESSFUL (total time: 0 seconds)
```

Type Casting: Widening vs. Narrowing

```
double d = 100.04;
long l = (long) d;// narrowing: explicit cast required
int  i = (int) l; // narrowing: explicit cast required

double f;
long g = 100;
f = g;          // widening: implicit, no cast needed
g = f;          // ERROR: possible loss of precision
```

- Widening casting (implicit): a smaller type is automatically promoted to a larger, more general one — int to long to float to double.
- Narrowing casting (explicit): converting to a smaller or less general type risks losing information, so Java forces you to write the cast, e.g. (int) l.
- Rule of thumb: going from a type with more information to one with less always needs an explicit cast.

Assigning a value of one type to a variable of another type is known as **Type Casting**. In Java, type casting is classified into **two** types,

❖Widening Casting(Implicit)



❖Narrowing Casting(Explicitly done)



Strings and Concatenation

```
String a = "hello" + 42;     // "hello42"
String b = 1 + "abc" + 2;    // "1abc2"
String c = "abc" + 9 * 3;    // "abc27"  (9*3 evaluated first)
String d = 1 + 2 + "abc";    // "3abc"   (1+2 evaluated first)
String e = "abc" + 4 - 1;    // COMPILE ERROR: cannot subtract from a String
```

- The + operator between a String and any other value converts that value to text and joins them.
- When both operands left of + are numbers, ordinary arithmetic happens first; a String only enters once + is used with a String operand.
- There is no “–” operator for String, which is exactly why the last line fails to compile.

1.5. Control Structures

Conditional statements: if, if–else, and the Ternary Operator

```
if (score >= 90) {
    grade = 'A';
} else if (score >= 80) {
    grade = 'B';
} else {
    grade = 'C';
}

// Ternary operator: a compact if-else that
// returns a value
String result = (score >= 50) ? "Pass" : "Fail";
```

- if executes a block only when its boolean condition is true.
- else if chains let you test several mutually exclusive conditions in order; the first true branch wins.
- else catches everything not matched above it, and is optional.
- The ternary operator condition ? a : b is an expression, not a statement — useful for short, single-value decisions.

Selection statements: switch-case

```
switch (x) {
    case 1:
        System.out.println("Got a 1");
        break;
    case 2:
    case 3:
        System.out.println("Got 2 or 3");
        break;
    default:
        System.out.println("Not a 1, 2, or 3");
        break;
}
```

- x must be a byte, short, char, int, an enum, or a String — never a long, float, double, or boolean.
- Falling through: case 2: with no break falls into case 3: — a deliberate way to group cases that share behavior.
- break exits the switch immediately; forgetting it is one of the most common Java bugs.
- default runs when no case matches; it's optional but good practice to include.

Loops

while — condition checked first

- while (condition) { statements; }
- The body may run zero times if the condition is false from the start.
- Example:

```
int number = 1;
while (number <= 200) {
        print(number);
        number *= 2;
}
```

- Output: 1 2 4 8 16 32 64 128

do–while — condition checked last

- do { statements; } while (condition);
- The body always runs at least once, even if the condition starts false.
- Example:

```
do {
        die = rand.nextInt(6)+1;
} while (die == 3);
```

- Use when you need “try once, then repeat while needed” logic — e.g. re-prompting a user.

The for Loop and the Enhanced for-each Loop

```
for (int i = 0; i < 10; i++) {
    System.out.println(i);
}

// Enhanced for-each (Java 5+) — reads more naturally over a collection
int[] numbers = {10, 20, 30, 40, 50};
for (int x : numbers) {
    System.out.print(x + ",");
}
```

- Classic for: for (start; test; update) bundles initialization, condition, and increment in one line — ideal when you need the index.
- for-each: for (type var : array) hides the index entirely — ideal when you only need each element's value, not its position.
- for-each cannot modify the underlying array through its loop variable, and cannot easily iterate backward or skip elements.

```
for (start_expr; test_expr;increment_expr){
   // code to execute repeatedly
}
```

- ```
  for (int index = 0; index < 10; index++) {
     System.out.println(index);
  }
  ```
- ```
  for (int i = -3; i <= 2; i++) {
      System.out.println(i);
  }
  ```
- ```
  for (int i = 3; i >= 1; i--) {
      System.out.println(i);
  }
  ```
  ```

Enhanced *for* Loops

- Java's for loops were enhanced in release 1.5 to work more easily with **arrays** and **collections**.
- Syntax: for (type variable_name:array)

```
int sumOfLengths(String[] strings) {
        int totalLength = 0;
        for (String s:strings)
                totalLength += s.length();
        return totalLength;
}
```

```
public class Example{
  public static void main(String args[]){
     int [] numbers = {10, 20, 30, 40, 50};
     for(int x : numbers ){
         System.out.print( x );// numbers[i]
         System.out.print(",");
     }
     System.out.print("\n");
    String [] names ={"James", "Larry", "Tom", "Lacy"};
     for( String name : names ) {
         System.out.print( name );
         System.out.print(",");
     }
  }
}
```

break, continue, and Labeled Loops

```
outer:
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        if (j == 3) continue outer;// skip to next i
        if (i == 4) break outer;// exit both loops entirely
        System.out.println(i + "," + j);
    }
}
```

- break exits the nearest enclosing loop (or switch) immediately.
- continue skips the rest of the current iteration and jumps to the loop's next test/update step.
- A label (outer:) placed before a loop lets break or continue target an outer loop directly from inside a nested one — without it, they only affect the innermost loop.

without label

```
for( .;.;.)
{
    //process part 1
    if(condition)
        continue;
    //process part 2
}
```

```
for( .;.;.)
{
    //process part 1
    if(condition)
        break;
    //process part 2
}
```

```
for (i=0; i<=5; ++i) {
    if (i % 2 == 0)
        continue;
    System.out.println("This is a " + i + " iteration");
}
```

```
i = 0;
while (i <= 5) {
    ++i;
    if (i % 2) == 0)
        continue;
    System.out.println("This is a odd iteration - " + i);
}
```

```
int i = 1;
while (true) {
    if (i = = 3)  break;
    System.out.println("This is a " + i + " iteration");
    ++i;
}
```

Choosing the Right Control Structure

Situation	Best Choice
You know exactly how many times to repeat, and need a counter.	for loop
You want to process every element of an array/collection by value only.	enhanced for-each loop
The number of repetitions depends on a condition evaluated up front.	while loop
The body must run at least once no matter what (e.g., menus, input validation).	do–while loop
You are branching on one variable against several discrete constant values.	switch statement
You are branching on a boolean expression, a range, or several variables at once.	if / else if / else

1.6. Standard Input and Output

Interactive programs

❖We have written programs that print console output.

❖It is also possible to read *input* from the console.

- The user types the input into the console.
- The program uses the input to do something.
- Such a program is called an *interactive program*.

Constructing a Scanner object to read the console:

Scanner ***<name>*** = new Scanner(System.in);

E.g. Scanner console = new Scanner(System.in);

console.next()

Method	Description
nextInt()	reads and returns user input as an int
nextDouble()	reads and returns user input as a double
next()	reads and returns user input as a String

Standard Output: print, println, and printf

```
System.out.print("No newline after this. ");
System.out.println("This line ends with a
newline.");
System.out.println();  // prints just a blank line

double gpa = 3.86;
System.out.printf("GPA: %.2f%n", gpa);// GPA: 3.86
```

- **print** writes text with no trailing newline; the next output continues on the same line.
- **println** writes text followed by a newline; calling it with no arguments just moves to the next line.
- **printf** gives C-style formatted output — %d for integers, %.2f for a decimal rounded to 2 places, %s for strings, %n for a platform-correct newline.

Standard Input: The Scanner Class

```
import java.util.Scanner;

Scanner console = new Scanner(System.in);
System.out.print("How old are you? ");//prompt
int age = console.nextInt();
System.out.println("You'll be 40 in " + (40 - age)
+ " years.");
```

- **Scanner** lives in the java.util package, so it must be imported before use.
  - Useful methods: **next()** a word, **nextLine()** a full line, **nextInt(), nextLong(), nextFloat(), nextDouble()**.
  - Each method blocks (pauses) the program until the user types something and presses Enter.
- A token is a unit of input separated by whitespace; if the next token doesn't match the type you asked for, Scanner throws an InputMismatchException and the program crashes.

❖ Input tokens

- **token**: A unit of user input, as read by the Scanner.
  - Tokens are separated by whitespace (spaces, tabs, new lines).
  - How many tokens appear on the following line of input?

    23 John Smith 42.0 "Hello world"

- When the token doesn't match the type the Scanner tries to read, the program crashes.
  - Example:

```
System.out.print("What is your age? ");
int age = console.nextInt();
```

Sample Run:

```
What is your age? Timmy
InputMismatchException:
  at java.util.Scanner.throwFor(Unknown Source)
  at java.util.Scanner.next(Unknown Source)
  at java.util.Scanner.nextInt(Unknown Source)
  ...
```

Importing classes

- **Java class libraries**: A large set of Java classes available for you to use.
  - Classes are grouped into *packages*.
  - To use the classes from a package, you must include an *import declaration* at the top of your program.
  - Scanner is in a package named java.util

- Import declaration, general syntax:

  import ***<package name>***.*;

- To use Scanner, put this at the start of your program:

  import java.util.*;

- public String next()
- public String nextLine()
- public byte nextByte()
- public short nextShort()
- public int nextInt()
- public long nextLong()
- public float nextFloat()
- public double nextDouble()

Classic Pitfall: Mixing nextInt() and nextLine()

```
System.out.println("Enter numerical value");
int option;
option = console.nextInt();
System.out.println("Enter 1st string");
String string1 = console.nextLine();
System.out.println("Enter 2nd string");
String string2 = console.nextLine();
```

```java
int option = input.nextInt();
console.nextLine();     // consume the leftover newline!
String name = console.nextLine(); // now reads correctly

// Alternative: read everything as a line, then parse
int option2 = Integer.parseInt(input.nextLine());
```

- nextInt() reads only the number, leaving the newline character behind in the input buffer.
- The very next nextLine() call then reads that leftover empty newline instead of the text you wanted.
- Fix #1: insert an extra input.nextLine() right after nextInt() purely to consume the leftover newline.
- Fix #2: read every line as a String with nextLine(), and parse numbers out of it yourself with Integer.parseInt() or Double.parseDouble().

Putting It All Together: A Small Interactive Program

```
import java.util.Scanner;

public class TemperatureConverter {
    public static void main(String[] args) {
        Scanner console = new Scanner(System.in);
        System.out.print("Enter a temperature in Celsius: ");
        double celsius = console.nextDouble();

        double fahrenheit = celsius * 9.0 / 5.0 + 32;
        System.out.printf("%.1f C is %.1f F%n", celsius, fahrenheit);
    }
}
```

- This one short program already combines everything in this chapter: class & main method structure, a double variable, an operator expression, standard input via Scanner, and formatted standard output.
- Notice 9.0 / 5.0, not 9 / 5 — using integer literals here would silently truncate to 1, an example of exactly the logic-error pitfall from Section 1.4.

InputOutputDemo.java

```
1  /* Write a program that will accept an array of intergers then
2     print out entered value and the sum of values
3   */
4  import java.util.Scanner;
5  public class InputOutputDemo {
6     public static void main (String args[])
7     {  int a[]; // array of integers
8        int n ;  // number of elements of the array
9        int i;   // variable for traversing the array
10       Scanner sc= new Scanner(System.in); // object for the keyboard
11       System.out.print("Enter number of elements: ");
12       n = Integer.parseInt(sc.nextLine());
13       a = new int[n]; // mem. allocating for elements of the array
14       for (i=0;i<n;i++)
15       {  System.out.print("Enter the " + (i+1) + "/" + n + " element: ");
16          a[i]=Integer.parseInt(sc.nextLine());
17       }
18       System.out.print("Entered values: ");
19       for (i=0;i<n;i++) System.out.format("%5d", a[i]);
20       int S=0;
21       for (int x: a) S+=x;
22       System.out.println("\nSum of values: " + S);
23    }
24 }
```

Output - Chapter01 (run) #2

```
run:
Enter number of elements: 5
Enter the 1/5 element: 1
Enter the 2/5 element: 4
Enter the 3/5 element: 2
Enter the 4/5 element: 0
Enter the 5/5 element: 7
Entered values:     1    4    2    0    7
Sum of values: 14
BUILD SUCCESSFUL (total time: 11 seconds)
```

documentation:
java.lang.String class,
- the **format** method,
- format string

Q&A

Choosing Your Project: What to Discuss With Your Team

Problem fit: does the idea naturally use the topics from this chapter — variables, control structures, and console I/O — without requiring knowledge you haven't learned yet?
Scope: can a small team realistically design, code, and test it within the semester timeline? Prefer a small idea done well over an ambitious idea left unfinished.
Data to manage: what will your program store and manipulate (student records, book catalog, quiz questions, expenses)?
User interaction: will the user interact through a simple text menu on the console for now, evolving toward a GUI later in the course?
Team roles: who owns which class or feature, and how will you integrate everyone's work before the deadline?



Deliverable: a one-page proposal naming your application, its core menu options, and the main data it will track — due to next class.

- Case study
  - Using simple menu
  - https://code.ptit.edu.vn/student/question (from 1 to 25)
  - More: https://codelearn.io/learning/java-fundamentals